M10: Package Management with APT: Installing Software

> "Your app needs Nginx, PostgreSQL, Redis, and monitoring tools. Install them all — correctly, securely, and with automatic updates."

Quick Check

Show of hands:

- Who has downloaded software from random websites?
- Who has wondered why Linux doesn't have an "install wizard"?
- Who has dealt with dependency hell?

> Today you'll learn the system that makes software installation trivial — the same package management philosophy used by npm, pip, Docker, and every modern development tool.

What You'll Walk Away With

By the end of this mission, you'll be able to:

- **Understand what packages are** and why they beat manual downloads
- **Update package lists** and upgrade installed software
- **Install and remove software** with proper cleanup
- **Search for packages** and read package information
- **Add third-party repositories** (and understand the risks)
- **Use dpkg** for low-level package management

Commands you'll learn: `apt update`, `apt upgrade`, `apt install`, `apt remove`, `apt purge`, `apt search`, `apt show`, `dpkg`

Your Server Is Secure, But Empty

So far we've covered:

- Users and groups → who can access
- sudo → what they can do
- SSH → how they connect

Next question: How do you install the software your server needs?

```
Need a web server?  → sudo apt install nginx
Need a database?    → sudo apt install postgresql
Need monitoring?    → sudo apt install htop
```

What Is a Package?

A **package** is a pre-built software bundle containing:

Component	Purpose
Binary files	The actual program
Configuration files	Default settings
Dependencies	Other packages it needs
Install scripts	Pre/post-install setup

Why not just download from websites?

Method	Packages (apt)	Manual download
Dependencies	Automatic	You figure it out
Updates	`apt upgrade`	Check each website
Security patches	Automated	Hope you notice
Uninstall	Clean removal	Files everywhere

apt update — Refresh Package List

What it does: Downloads latest list of available packages. Does NOT install anything.

Syntax: `sudo apt update`

```
sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Fetched 229 kB in 1s
15 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Think of it like this:

- `apt update` = Refreshing the menu at a restaurant
- `apt install` = Actually ordering food

> **Rule:** ALWAYS run `apt update` before `apt install`. The package list might be stale!

apt upgrade — Upgrade Installed Packages

What it does: Upgrades all installed packages to newest versions.

Syntax: `sudo apt upgrade [-y]`

```
# See what would be upgraded
apt list --upgradable

# Upgrade all packages (asks confirmation)
sudo apt upgrade

# Upgrade without asking (scripts)
sudo apt upgrade -y
```

Upgrade command differences:

Command	What It Does
`apt upgrade`	Upgrades, never removes
`apt full-upgrade`	Upgrades + removes if needed

apt install — Installing Packages

What it does: Downloads and installs package with dependencies.

Syntax: `sudo apt install [-y] package [package2 ...]`

```
# Install single package
sudo apt install nginx

# Install without confirmation
sudo apt install -y nginx

# Install multiple packages
sudo apt install -y nginx curl htop tree

# Install specific version
sudo apt install nginx=1.18.0-6ubuntu14
```

Useful options:

Option	Purpose
`-y`	Answer "yes" to all prompts
`--no-install-recommends`	Skip non-essential packages
`--dry-run`	Show what would happen

apt remove vs apt purge — Removing Packages

Two ways to remove software:

```
# Remove package (keeps config files)
sudo apt remove nginx

# Remove package AND config files
sudo apt purge nginx

# Clean up unused dependencies
sudo apt autoremove
```

Command	Removes Binary	Removes Config	Removes Dependencies
`apt remove`	Yes	**No**	No
`apt purge`	Yes	**Yes**	No
`apt autoremove`	No	No	**Yes (unused)**

apt search / apt show — Finding Packages

`apt search` — **Find packages by keyword:**

```
apt search web server
nginx/jammy 1.18.0-6ubuntu14.3 amd64
  small, powerful, scalable web/proxy server
apache2/jammy 2.4.52-1ubuntu4.6 amd64
  Apache HTTP Server
```

`apt show` — **Detailed package info:**

```
apt show nginx
Package: nginx
Version: 1.18.0-6ubuntu14.3
Installed-Size: 601 kB
Depends: nginx-core | nginx-full
Description: small, powerful, scalable web/proxy server
```

Where Software Comes From — Repositories

Repository = A server hosting packages (like an app store)

```
# See configured repositories
cat /etc/apt/sources.list
deb http://archive.ubuntu.com/ubuntu jammy main restricted
deb http://archive.ubuntu.com/ubuntu jammy-updates main
deb http://security.ubuntu.com/ubuntu jammy-security main
```

Ubuntu repository categories:

Repository	Contents	Supported By
`main`	Free software, official	Canonical
`universe`	Free, community	Community
`restricted`	Proprietary drivers	Canonical
`multiverse`	Non-free software	Community

PPAs — Third-Party Repositories

PPA = Personal Package Archive (hosted on Launchpad)

```
# Add a PPA
sudo add-apt-repository ppa:ondrej/php
sudo apt update
sudo apt install php8.2

# Remove a PPA
sudo add-apt-repository --remove ppa:ondrej/php
```

When to use PPAs:

- Need newer software than official repos
- Software not in Ubuntu repos

Risks of PPAs:

- Not officially supported
- Could break system upgrades
- Maintainer could stop updating

> **Production rule:** Avoid PPAs. Use official repos or vendor repos (Docker, PostgreSQL, etc.).

dpkg — Low-Level Package Management

What it does: Manages individual `.deb` files directly (no dependency resolution).

```
# Install downloaded .deb file
sudo dpkg -i google-chrome-stable.deb

# If dependencies missing:
sudo apt install -f

# List all installed packages
dpkg -l | head -20

# List files in a package
dpkg -L nginx

# Find which package owns a file
dpkg -S /usr/sbin/nginx
nginx-core: /usr/sbin/nginx
```

APT Command Reference

Command	Purpose	Needs sudo?
`apt update`	Refresh package lists	Yes
`apt upgrade`	Upgrade installed packages	Yes
`apt install <pkg>`	Install package	Yes
`apt remove <pkg>`	Remove (keep config)	Yes
`apt purge <pkg>`	Remove + delete config	Yes
`apt autoremove`	Clean unused deps	Yes
`apt search <keyword>`	Find packages	No
`apt show <pkg>`	Package details	No

> **The golden rule:** Always `apt update` before `apt install`!

Aha! Moment 💡

Think about what apt solved in 2000:

1. **Dependency hell** — software needs other software
2. **Update nightmare** — checking 50 websites for patches
3. **Security patches** — how do you know there's an update?
4. **Clean uninstall** — files scattered everywhere

Now look at modern tools:

Tool	Language/Platform	Same Concept
`apt`	Linux packages	Original inspiration
`npm`	Node.js	`npm install`, `npm update`
`pip`	Python	`pip install`, `pip freeze`
`brew`	macOS	Literally uses apt concepts
`cargo`	Rust	Package registry + dependencies
`Docker`	Containers	Images = packages with dependencies

The universal workflow:

```
# apt pattern:
apt update && apt install nginx

# npm equivalent:
npm update && npm install express

# pip equivalent:
pip install --upgrade pip && pip install flask

# Docker equivalent:
docker pull nginx:latest
```

> apt didn't just solve Linux package management — it **invented the pattern** every modern development tool uses.

APT Package Management Cheat Sheet

Essential workflow:

```
# 1. ALWAYS update package lists first
sudo apt update

# 2. Upgrade all installed packages
sudo apt upgrade

# 3. Install new software
sudo apt install package-name

# Full update sequence (common pattern)
sudo apt update && sudo apt upgrade -y
```

Installing packages:

```
# Install single package
sudo apt install nginx

# Install without confirmation (scripts)
sudo apt install -y nginx

# Install multiple packages
sudo apt install nginx postgresql redis-server

# Install specific version
sudo apt install nginx=1.18.0-6ubuntu14

# Reinstall package (fix corruption)
sudo apt install --reinstall nginx

# Install without recommended packages (minimal)
sudo apt install --no-install-recommends nginx

# Dry run (show what would happen)
sudo apt install --dry-run nginx
```

Removing packages:

```
# Remove package (keep config files)
sudo apt remove nginx

# Remove package AND config files
sudo apt purge nginx

# Remove unused dependencies
sudo apt autoremove

# Full cleanup
sudo apt purge nginx && sudo apt autoremove
```

Searching and information:

```
# Search for packages
apt search web server
apt search ^nginx$          # Exact match

# Show package details
apt show nginx

# List upgradable packages
apt list --upgradable

# List installed packages
apt list --installed

# List all versions of package
apt list -a nginx
```

Upgrading:

```
# Upgrade all (safe - won't remove)
sudo apt upgrade

# Upgrade all (may remove if needed)
sudo apt full-upgrade

# Upgrade without prompts
sudo apt upgrade -y

# Upgrade specific package
sudo apt install --only-upgrade nginx
```

Cache management:

```
# Clear downloaded package files
sudo apt clean

# Clear old versions only
sudo apt autoclean
```

dpkg (low-level):

```
# Install .deb file
sudo dpkg -i package.deb

# Fix broken dependencies after dpkg
```

Mission Complete: Package Management ✅

You mastered software installation:

01

Understood what packages are and why they matter

02

Refreshed package lists with apt update

03

Upgraded software with apt upgrade

04

Installed packages with apt install

05

Removed cleanly with apt remove/purge

06

Searched for packages with apt search

07

Managed low-level packages with dpkg

Commands you now know: `apt update`, `apt upgrade`, `apt install`, `apt remove`, `apt purge`, `apt autoremove`, `apt search`, `apt show`,

> **Next mission:** Software is installed. But it's just sitting on disk. A web server needs to RUN continuously. A database needs to START on boot. A crashed service needs to RESTART automatically. How do you make software run — now and forever? Enter systemd — the service manager that keeps your server alive.

Questions?